

第一章 软件工程概述

本章速览

核心概念：软件（程序+数据+文档） | 软件危机 | 软件工程（方法/工具/过程） | SWEBOK 知识体系

核心问题：

1. 软件是什么？它与硬件有哪些本质区别？
2. 软件危机的表现、原因、解决途径分别是什么？
3. 软件工程如何定义？三要素、目标、原则、原理分别是什么？
4. SWEBOK 知识体系包含哪些知识域（KA）？

必背 七大高频考点：

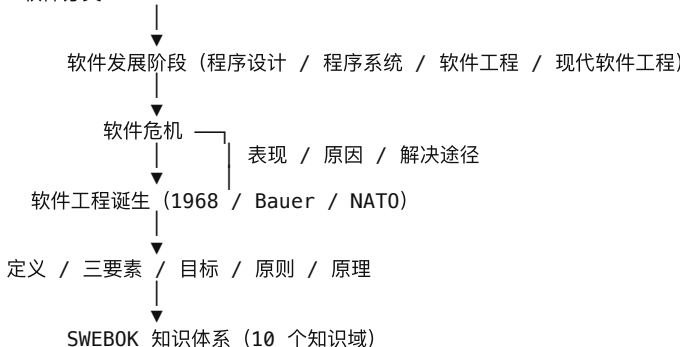
1. 软件 IEEE 定义：程序 + 规程 + 文档 + 数据
2. 软件特点 8 条（重点是"逻辑实体 / 无磨损但会退化 / 复杂性"）
3. 软件危机 7 大表现
4. 1968 年 Fritz Bauer 提出"软件工程"，三要素 = 方法 + 工具 + 过程
5. 软件工程 6 项基本活动（制定计划 / 需求 / 设计 / 编码 / 测试 / 维护）
6. 4 条基本原则 + 8 条一般原理 + Boehm 7 条基本原理
7. SWEBOK 10 个知识域

题目关键词导航

关键词 / 题干特征	对应知识点
"程序 + 数据 + 文档"	软件定义（IEEE / 公认解释）
"逻辑实体"、"抽象性"	软件特点 1
"退化"、"锯齿形曲线"	软件特点 3（与硬件磨损曲线对比）
"1968"、"Fritz Bauer"、"NATO"	软件工程诞生
"方法、工具、过程"	软件工程三要素
"OS/360"、"5000 人年"	软件危机典型案例
"Boehm 7 原理"、"少而精"、"阶段评审"	软件工程 7 条基本原理
"SWEBOK"、"10 个知识域"、"KA"	软件工程知识体系
"6 项基本活动"、"制定计划"、"编码"	软件生命周期活动
" $n(n-1)/2$ "、"通信信道"	Boehm 第 6 原理（少而精）

知识主线

软件定义 → 软件特征 → 软件分类



核心知识点详解

1. 软件的定义

IEEE 定义：软件是计算机**程序、规程**，以及运行计算机系统可能需要的相关**文档和数据**。

公认解释：软件是包括**程序、数据及其相关文档**的完整集合。

三个等式：

- 结构化程序设计：程序 = 算法 + 数据结构
- 软件工程视角：软件 = 程序 + 文档
- 完整集合：软件 = 程序 + 数据 + 文档

程序和数据是构造软件的基础；文档是软件质量的保证，也是保证软件更新及生命周期长短的必需品。

考点提示 填空题高频："软件是包括（程序）、数据和相关文档的完整集合"。

2. 软件的特点（8 条）

1. **逻辑实体，具有抽象性**：存储于磁盘/内存等介质，无法直接看到形态，必须通过观察、分析、思考或运行使用才能了解其功能、性能。
2. **无明显的制造过程**：软件开发是"创作"过程；研制成功后制造 = 复制。质量控制必须和研制过程交织在一起。
3. **不存在机械磨损和老化问题，但存在软件退化问题**。详见下方对比表。
4. **受计算机系统约束**：开发和运行都受限于硬件平台、操作系统等。

5. **未完全摆脱手工艺开发方式**：大部分软件是"定制"的，难以完全通过组装完成。
6. **高度复杂**：来源于**实际需求复杂** + **程序逻辑复杂**。两重复杂性：感性认识复杂性、理性认识复杂性。
7. **研制成本高**：软件成本在计算机系统总成本中所占比例逐年增加。
8. **涉及社会因素**：投入运行时还涉及许多社会因素（管理、文化、法律等）。

考点提示 第 1、2、3、6 条最常出现在判断/简答题中。常见陷阱题："软件是一种逻辑实体，由可执行代码构成"——错（缺少数据与文档）。

3. 软件的分类

3.1 按服务对象

- **通用软件** (Generic)：操作系统、数据库、字处理等；面向市场用户公开销售；可近零成本复制分摊开发成本。
- **定制软件** (Customized)：企业 ERP、OA 等；按客户合同以项目方式开发；仅服务一个客户，成本较高。
- **可配置软件** (Configurable)：本身具备完善功能，由客户按业务特点配置后使用；广泛存在工作流引擎、报表引擎、规则引擎；大型企业级应用主流方案。

3.2 按功能层次

- **系统软件**：与硬件紧密配合，使各部件、相关软件和数据协调、高效地工作。例：操作系统、设备驱动程序、数据库管理系统、编译程序、通信处理程序。
- **中间件软件**：位于操作系统和应用软件之间，管理分布式计算资源和网络通信；提供标准程序接口和协议。例：基础中间件、业务中间件、领域中间件。
- **应用软件**：在特定领域内开发，为特定目的服务。例：办公软件、CAD/CAM、行业软件 (BOSS/ERP/CRM/FMS/HIS/TMS)。

3.3 教材补充分类

- **按使用方式**：单机软件 / 服务器软件 / 客户端软件；SaaS (Software-as-a-Service，软件即服务)。
- **按工作方式**：实时处理软件 / 分时软件 / 交互式软件 / 批处理软件。
- **按规模 (LOC)**：微型 / 小型 / 中型 / 大型 / 甚大型 / 极大型 (详见补充知识点)。

4. 软件发展阶段（四阶段）

阶段	年代	特征
程序设计	50—60 年代	个体手工；汇编/机器语言；程序 = 算法+数据结构；只有程序概念
程序系统	60—70 年代	小组软件作坊；高级语言；结构化程序设计；软件 = 程序+说明书；出现软件危机
传统软件工程	70—90 年代	工程化思想；CASE 工具；面向对象兴起 (Smalltalk、C++)；CMM/COCOMO
现代软件工程	90 至今	Internet/Web；分布式、构件复用 (EJB/J2EE、COM+/DNA、CORBA/OMA)；面向服务

5. 软件危机

5.1 定义

软件危机 (Software Crisis)：计算机软件在开发和维护过程中所遇到的一系列严重问题，导致软件行业的信任危机。本质是落后的软件生产方式无法满足迅速增长的计算机软件需求。

5.2 典型案例

- **IBM OS/360 系统**：4000+ 模块、约 100 万条指令、人工 5000 人年、耗费数亿美元；投入运行后发现 2000+ 错误，每个版本更新仍有 1000+ 错误，开发陷入僵局。
- **火星探测器爆炸** (1963 年)：控制软件中一个"，"号被误写为"."，致使飞往火星的探测器发生爆炸，损失数亿美元。

5.3 七大表现（必背）

1. **开发计划难制订**：成本、进度估算不准，预算超支，期限一拖再拖。
2. **费用和进度失控**：超支、拖延频发，赶进度损害质量。

3. **产品无法让用户满意**：初期需求不明确，开发中沟通不足。
4. **质量难保证**：审查、复审、测试等质量保证技术未持续应用。
5. **不可维护**：错误难改、改后引入新错误，难增加新功能。
6. **缺乏适当文档**：文档不完整或与代码不一致。
7. **成本占比逐年上升**：硬件成本下降，软件成本逐年上升。

5.4 产生原因

内在原因（两方面）：

- **软件本身的复杂性**——规模庞大、复杂度随规模指数上升；逻辑产品、依赖高度智力投入。
- **软件开发和维护方法不合理、技术不成熟**——早期过度推崇个人技巧，导致可读性、可维护性差。

深层原因：

- 缺乏对需求准确刻画的工具、缺少需求确认环节、客户需求变化未及时变更；
- 系统架构设计不合理 → 软件无法扩充；
- 缺乏有效测试环节 → 上线后错误不断；
- 缺乏可移植性考虑 → 难以跨环境运行。

5.5 解决途径

- **1968 年原联邦德国 NATO 会议首次提出"软件工程"概念。**
- 提出运用**工程化原则和方法**组织软件开发，从根本上摆脱落后的手工作坊式生产方式。

6. 软件工程的定义

提出者	定义
Fritz Bauer	为了 经济地 获得能够在实际机器上高效运行的 可靠软件 而建立和使用的一系列好的 工程化原则 (1968)。
Boehm	运用现代科学技术知识来设计并构造计算机程序及为开发、运行和维护这些程序所必需的相关文件资料。
Fairley	在成本限额以内按时完成开发和修改软件产品所需系统生产和维护的技术和管理的学科。
IEEE	① 应用 系统化、规范化、定量 的方法来开发、运行和维护软件，即 将工程应用到软件 ；② 对①中各种方法的研究。

核心思想：在软件开发过程中应用工程化原则。

两重要内容：

1. 软件工程是工程概念在软件领域的特定应用，需选择和应用适当理论、方法和工具。
2. 软件工程涉及软件产品的所有环节——**项目管理失败的危害基于技术失败。**

7. 软件工程**三要素**（必考）

要素	作用
方法 Methods	提供"如何做"的技术——项目计划、需求分析、设计、编码、测试、维护等各阶段。
工具 Tools	提供自动或半自动的软件支撑环境——CASE（计算机辅助软件工程）。
过程 Process	将方法和工具综合起来以达到合理、及时地进行软件开发的目的。定义方法使用顺序、文档交付要求、质量保证和变更管理。

考点提示 "软件工程三要素是方法、工具、过程"——记忆诀窍：方工过。常见陷阱题："三要素是程序、数据、文档"——错（那是软件的定义）。

8. 软件工程的**目标**

总目标：在给定**成本、进度**的前提下，开发出满足用户需求且具有下列 10 个属性的软件产品：**软件工程项目三个基本目标：合理的进度、有限的经费、一定的质量。**

1. **可修改性** Modifiability
2. **有效性** Efficiency（时空资源最有效利用）
3. **可靠性** Reliability
4. **可理解性** Understandability
5. **可维护性** Maintainability
6. **可重用性** Reusability
7. **可适应性** Adaptability
8. **可移植性** Portability
9. **可追踪性** Traceability
10. **可互操作性** Interoperability

最终目的：摆脱手工生产软件的状况，逐步实现软件研制和维护的自动化。

9. 软件工程研究内容

- **软件开发技术：**开发方法学、开发过程模型、开发工具、软件工程环境（主体是开发方法学）。
- **软件工程管理：**软件管理学（人员组织、进度安排、质量保证、配置管理、项目计划）、软件工程经济学（成本估算、效益分析）、软件心理学。

10. 软件工程的 4 条基本原则

1. **选取适宜的开发模型：**认识需求的易变性，采用适宜模型予以控制（与系统设计有关）。
2. **采用合适的设计方法：**考虑模块化、抽象与信息隐藏、局部化、一致性、适应性。
3. **提供高质量的工程支持力度：**软件工具与环境对过程的支持颇为重要。
4. **重视开发过程的管理：**直接影响可用资源的有效利用、生产能力。

11. 软件工程原理

11.1 一般原理（8 条）

1. **抽象：**抽取事物最基本的特性和行为，自顶向下、逐层细化。
2. **信息隐藏：**封装为"黑箱"，仅通过模块接口访问；访问与实现分离。
3. **模块化：**模块大小适中——过大增复杂度，过小致系统碎片化。
4. **局部化：**物理模块内集中逻辑关联资源，松耦合、强内聚。
5. **确定性：**概念表达无歧义、规范，避免交流误解。
6. **一致性：**整个系统使用一致的概念、符号、术语、接口。
7. **完备性：**不丢失重要成分，可完全实现系统要求功能；须严格技术评审。
8. **可验证性：**自顶向下逐层分解，便于检查、测试、评审。

11.2 Boehm 7 条基本原理（必背）

1. **用分阶段的生命周期计划严格管理：**6 类计划（项目概要、里程碑、项目控制、产品控制、验证、运行维护）；50%+ 失败项目因计划不周。
2. **坚持进行阶段评审：**63% 错误在编码之前造成；错误发现越晚代价越大。
3. **实行严格的产品控制：**基线配置管理（变更控制），需求变动时各阶段文档/代码同步变更。
4. **采用现代程序设计技术：**结构化 → 面向对象，提高效率、降低维护成本。
5. **结果应能清楚地审查：**明确开发小组的责任和产品标准。
6. **开发小组的人员应少而精：**高素质人员效率高数倍至数十倍；通信信道数 = $n(n-1)/2$ ，人数越多通信开销急剧增大。
7. **承认不断改进软件工程实践的意义：**积极采纳新技术，总结经验，收集数据。

考点提示 记忆诀窍：计评控技查少进（计划-评审-控制-技术-审查-少而精-改进）。

12. SWEBOK 软件工程知识体系

12.1 起源与目的

SWEBOK（Guide to Software Engineering Body of Knowledge，软件工程知识体系指南）：由 IEEE 计算机学会专业实

践委员会主持的项目，1998—1999 年启动；2004 年发布"铁人版本"作为标准指南。

三大阶段：草人阶段（雏形）→ 石人阶段（试用版）→ 铁人阶段（2004 标准）。

5 项目：

1. 促进世界范围内对软件工程的一致观点；
2. 阐明软件工程相对其他学科（计算机科学、项目管理、计算机工程、数学等）的位置并确立分界；
3. 刻画软件工程学科的内容；
4. 提供使用知识体系的主题；
5. 为开发课程表和个人认证与许可材料提供基础。

12.2 10 个知识域（KA, Knowledge Area）

开发主流程（5 个）	支撑性活动（5 个）
软件需求 Software Requirements	软件配置管理 SCM
软件设计 Software Design	软件工程管理 SEM
软件构造 Software Construction	软件工程过程 SEP
软件测试 Software Testing	软件工程工具和方法 SETM
软件维护 Software Maintenance	软件质量 SQ

考点提示 记忆诀窍：前 5 个是"开发主流程"（需求→设计→构造→测试→维护），后 5 个是"支撑性活动"（配置管理 / 工程管理 / 工程过程 / 工具方法 / 质量）。

12.3 相关学科

计算机工程、计算机科学、管理、数学、项目管理、质量管理、软件人类工程学、系统工程。

13. 软件工程的 6 项基本活动（必考） 在第二章 软件生命周期中

1. 制定计划（含可行性分析、系统分析）
2. 需求分析（需求定义）
3. 软件设计（概要设计 + 详细设计）
4. 程序编写 / 编码
5. 软件测试
6. 运行和维护

14. 图示信息：硬件失效曲线 vs 软件退化曲线

图示信息

主题：硬件失效率与软件失效率随时间变化的对比。

横轴：时间。 **纵轴：**失效率（Failure Rate）。

硬件曲线（浴盆曲线）：初期高（机械电子器件磨合问题）→ 中期低且稳定（正常运行）→ 末期急剧上升（磨损老化达到生命周期终点，彻底失效）。

软件曲线：

- **理想曲线：**单调下降，随错误修复趋近于零。
- **实际曲线（退化曲线）：**初期高，逐步下降稳定 → 每次修改点失效率再次跃升（修改引入新错误） → 长期反复修改后稳定性和可维护性越来越差 → 软件退化，最终被废弃。

核心结论：

- 硬件：**有磨损、寿命终点一次性失效**（浴盆曲线）。
- 软件：**无机械磨损，但存在退化现象**（锯齿形曲线）——这是软件工程的核心特征之一。

补充知识点（教材独有）

补 1：LOC 软件规模分类表（表 1-2）

类别	参加人员数	开发周期	产品规模（LOC）
微型	1	1—4 周	0.5k
小型	1	1—6 月	1k—2k
中型	2—5	1—2 年	5k—50k
大型	5—20	2—3 年	50k—100k
甚大型	100—1000	4—5 年	1M（=1000k）
极大型	2000—5000	5—10 年	1M—10M

补 2：与软件工程相关的学科（表 1-3）

计算机工程	Computer Engineering	项目管理	Project Management
计算机科学	Computer Science	质量管理	Quality Management
管理	Management	软件人类工程学	Software Ergonomics
数学	Mathematics	系统工程	System Engineering

补 3：可配置软件的底层工具

广泛存在**工作流引擎、报表引擎、规则引擎**等底层工具以支持软件的灵活配置。可配置软件提高了复用率，满足企业对软件灵活性的

要求，已成为大型企业级应用的主流开发方案。

补 4：CASE 工具

CASE (Computer Aided Software Engineering, 计算机辅助软件工程)：将各种软件工具、开发机器和一个存放开发过程信息的工程数据库组合而形成一个软件工程环境。

补 5：SaaS（软件即服务）

服务器平台直接向用户提供软件服务。SaaS 提供商为用户搭建信息化所需要的所有网络基础设施及软件、硬件运作平台，并负责所有前期的实施、后期的维护等一系列服务。用户根据实际需要，从 SaaS 提供商租赁软件服务。

核心对比表

对比 1：硬件失效 vs 软件退化（必考）

维度	硬件	软件
实体属性	物理实体	逻辑实体（抽象）
失效原因	机械电子器件磨损、老化	修改引入新错误、需求变化
失效曲线	浴盆曲线（初高→稳定→未高）	锯齿形（修改点跃升）
寿命终点	一次性失效、彻底报废	维护费用过高或不能满足需求时废弃
维修方式	替换零件	修改代码
制造过程	批量制造、可控制质量	无制造过程、复制即部署
质量控制	制造过程中控制	研制过程中控制

对比 2：系统软件 vs 中间件 vs 应用软件

类型	位置	主要功能	典型例子
系统软件	最底层，紧贴硬件	协调各部件、软硬件协同高效工作	操作系统、设备驱动、DBMS、编译程序
中间件	操作系统与应用之间	管理分布式计算资源和网络通信	基础中间件、业务中间件、领域中间件
应用软件	顶层，特定领域	为特定目的服务	办公软件、CAD/CAM、行业软件

开放题答题模板

模板 1：为什么软件没有磨损但会退化？（必考）

1. 软件是逻辑实体，没有机械部件——所以不存在物理磨损和老化。

2. 软件退化的原因：

软件在修改时可能引入新错误，使失效率在修改点升高；

用户需求不断变化，经过反复修改的软件稳定性和可维护性越来越差；

软件功能与实际业务匹配度越来越低。

3. 结论：软件失效率呈"锯齿形"曲线，每一次修改都可能带来新的失效高峰。当软件已无法满足用户需求或维护费用过高时，软件被废弃——这就是软件的"退化"。

补 6：国家软件分类编码（节选）

编码	类别	编码	类别
10000	基础软件（总分类）	30109	信息安全软件
10100	操作系统	30200	行业应用软件
10200	数据库系统	30300	文字语言处理
20000	中间件（总分类）	40000	嵌入式应用软件
30000	应用软件（总分类）	...	...

对比 3：通用 vs 定制 vs 可配置软件

维度	通用软件	定制软件	可配置软件
服务对象	市场用户	特定客户	多个相似企业
成本分摊	近零成本复制	高（单客户负担）	中等（基于复用）
灵活性	低（固定功能）	高（按需开发）	高（通过配置）
维护方式	开发组织统一升级	合约维护	重新配置
典型例子	操作系统、字处理	企业 ERP 项目	OA、CRM、HR 系统

对比 4：4 基本原则 vs 8 一般原理 vs 7 基本原理

类别	数量	性质	要点
基本原则	4 条	实践层面	模型 / 方法 / 支持 / 管理
一般原理	8 条	设计原理	抽象 / 隐藏 / 模块化 / 局部化 / 确定性 / 一致性 / 完备性 / 可验证性
基本原理 (Boehm)	7 条	工程层面	计划 / 评审 / 控制 / 技术 / 审查 / 人员 / 改进

对比 5：软件工程 vs 程序设计

维度	程序设计	软件工程
对象	程序	软件（程序+数据+文档）
方式	个体手工	工程化、团队协作
范围	编写代码	软件生命周期全过程
质量决定因素	个人程序技术	管理水平
关注点	算法正确性	可维护性、可靠性、可重用等

模板 2：什么是软件危机？表现、原因、解决途径？

1. 定义：计算机软件在开发和维护过程中所遇到的一系列严重问题，导致软件行业的信任危机。本质是落后的软件生产方式无法满足迅速增长的软件需求。

2. 7 大表现：① 开发计划难制订；② 费用进度失控；③ 产品无法让用户满意；④ 质量难保证；⑤ 不可维护；⑥ 缺乏适当文档；⑦ 成本占比逐年上升。

3. 原因：

内在：软件本身复杂性 + 软件开发方法和技术不合理、不成熟；

深层：早期过度推崇个人技巧；缺乏需求准确刻画工具；架构不合理；缺乏有效测试；缺乏可移植性考虑。

4. 解决途径：1968 年 NATO 会议 Fritz Bauer 首次提出"软件工程"概念，运用工程化原则和方法组织软件开发。

模板 3：如何理解软件工程是为解决软件危机而产生的？

1. 背景：20 世纪 60 年代，软件危机爆发（OS/360 等典型失败案例）。
2. 需求：落后的软件生产方式（手工作坊）无法满足迅速增长的软件需求。
3. 诞生：1968 年 NATO 会议上 Fritz Bauer 首次提出"软件工程"——将工程化原则应用于软件。
4. 作用：

提供"方法、工具、过程"三要素系统化指导；

提出 4 条基本原则（模型、方法、支持、管理）；

提出 8 条一般原理和 Boehm 7 条基本原理；

SWEBOK 形成完整知识体系。
5. 现实意义：软件工程并非"银弹"，至今未完全解决软件危机，但显著提高了开发效率和软件质量。

模板 4：软件工程方法是否能解决所有软件开发问题？

1. 承认局限：软件工程显著提高了开发效率和质量，但未能完全解决软件危机——这是行业共识。
2. 原因：

软件本身复杂性持续增长；

用户需求不断变化；

新技术（AI、大数据、云原生）带来新挑战；

团队管理和人员因素仍不可控。
3. 结合实践：（例如课程作业经验）即使在小组作业中，也会遇到需求不清、协作困难、版本冲突、测试不足等问题。
4. 改进方向：不断采纳新技术，遵循 Boehm 第 7 条原理——承认不断改进软件工程实践的意义。

模板 5：软件项目失败的可能原因？

1. 开发进度失控：不能在规定时间内完成项目开发；
2. 开发成本过高：预算超支，导致软件项目无法收回成本；
3. 可维护性差：所开发项目缺乏可维护性，不能适应变化的需求，很快夭折；
4. 可补充：需求不明确、缺乏文档、团队沟通不畅（通信开销 $n(n-1)/2$ ）、管理方法不当、缺乏适当的开发模型等。

模板 6：软件工程产生的历史背景及作用（2023–2024 真题）

- 历史背景：

1. 20 世纪 60 年代软件规模扩大、复杂度提高，落后的手工作坊式开发方式无法满足需求；

2. 软件危机爆发（IBM OS/360、火星探测器等典型案例）；

3. 1968 年原联邦德国 NATO 会议，Fritz Bauer 首次提出"软件工程"概念。
- 作用：

1. 系统化指导：通过方法、工具、过程三要素系统化组织开发；

2. 规范化活动：定义了 6 项基本活动贯穿软件生命周期；

3. 质量保证：通过 4 条基本原则、8 条一般原理、Boehm 7 条基本原理保障质量；

4. 知识体系：SWEBOK 形成完整的知识体系，支持人才培养和职业认证；

5. 缓解危机：显著缓解软件危机（但未完全解决）。

历年真题汇总与详解

一、判断题

- 【2007–2008 期末 A】1. 软件是一种逻辑实体，由可执行代码构成。（）
答案：×。软件是逻辑实体正确，但 IEEE 定义明确：软件包括程序、规程、相关文档和数据，并非仅由"可执行代码"构成。文档和数据是软件不可或缺的部分。
- 【2008–2009 期末 A】1. 软件使用很长一段时候后会出现退化问题。（）
答案：√。这是软件区别于硬件的核心特征之一。软件无机械磨损，但反复修改会引入新错误，需求变化也使稳定性和可维护性越来越差，最终退化。
- 【2009–2010 期末 A】1. 软件是一种逻辑实体，由文档和可执行代码构成。（）
答案：×。软件是逻辑实体正确，但构成应包括"程序、数据及其相关文档"完整集合，缺少"数据"，且"可执行代码"不严谨（还应包括规程）。
- 【2009–2010 期末 A】4. 软件使用很长一段时候后会出现退化问题。（）
答案：√。同上。
- 【2010–2011 期中 A · 有答案】1. 缺乏处理大型软件项目的经验，是产生软件危机的唯一原因。（×）
"唯一原因"过于绝对。软件危机的产生有多重原因：软件本身的复杂性、软件开发方法和技术不合理、缺乏适当管理、需求变化等。
- 【2010–2011 期中 B】1. 软件的定义可以表述为程序与文档的集合。（）
答案：×。公认解释为"软件是包括程序、数据及其相关文档的完整集合"，缺少"数据"。

- 【2010–2011 期中 B】2. 软件的复杂性主要体现在程序逻辑的复杂性和软件规模上。（）
答案：√。教材明确指出软件复杂性的两重来源——实际需求（业务背景）的复杂性、程序逻辑的复杂性；规模庞大也使复杂度随规模指数上升。
- 【2010–2011 期中 B】3. 软件工程的出现是解决软件危机的关键。（）
答案：√。1968 年 NATO 会议提出软件工程概念，正是为解决软件危机而生。
- 【2010–2011 期中 B】4. 软件工程三要素是方法、过程和人的技术能力。（）
答案：×。软件工程三要素是方法、工具、过程。"人的技术能力"不属于三要素。
- 【2010–2011 期中 B】6. 软件工程的最终目的是摆脱手工生产软件的状况，逐步实现软件研制和维护的自动化。（）
答案：√。教材原文，PPT 也有相同表述。
- 【2010–2011 期末 B】1. 软件工程的主要思想是强调在软件开发过程中需要应用工程化的原则。（）
答案：√。所有软件工程定义（Bauer、Boehm、Fairley、IEEE）均强调"将工程应用到软件"，工程化原则是核心思想。
- 【2011–2012 期末 B · 有答案】1. 导致软件项目失败的主要原因是采纳的技术和工具，而管理过程是次要的。（×）
教材原文："统计数据表明，导致软件项目失败的主要原因并不是采纳的技术和工具，而是不适当的管理方法。"
- 【2012–2013 期末 B · 有答案】10. 软件的项目管理就是软件工程。（×）
项目管理只是软件工程三要素中"过程"的一部分（涉及开发进度、配置管理等）。软件工程还包括方法、工具以及大量技术活动（分析、设计、编码、测试）。

【2015-2016 期中 A · 有答案】1. 缺乏软件项目管理的经验，是产生软件危机的原因之一。(Y)

教材指出多数软件开发项目失败并非技术原因，而是不适当的管理方法。软件危机的多重原因中包括管理不当。

【2015-2016 期中 A · 有答案】4. 时至今日软件工程还未能解决软件危机中的所有问题。(Y)

软件工程显著缓解了软件危机，但未完全解决——这是行业共识。

【2015-2016 期中 A · 有答案】5. 软件工程为软件开发提供了如何做开发和管理的技术。(Y)

方法提供了"如何做"的技术，过程定义了"如何综合应用"，软件工程确实为开发和管理提供技术。

【2018-2019 期末 B】1. 软件工程的三要素是指程序、数据及相关文档。(x)

软件工程的三要素是方法、工具、过程。"程序、数据及相关文档"是软件的定义，不是软件工程三要素。

【2023-2024 期末 A】1. 软件工程起源于解决程序设计不能解决的问题。()

答案：√。软件工程起源于 20 世纪 60 年代软件危机——程序设计阶段（个体手工方式）无法应对规模庞大、复杂度高的软件系统，由此诞生软件工程。

【2024-2025 期中 A】1. 软件工程是一门随着新技术的出现而不断发展和演进的学科。()

答案：√。教材与 Boehm 第 7 条原理"承认不断改进软件工程实践的意义"完全一致。从结构化 → 面向对象 → 构件化 → 服务化 → 智能化，软件工程持续演进。

二、单项选择题

【2008-2009 期末 A】1. 导致软件危机的最主要原因是 ()

A. **开发方法和技术不合理** B. 软件需求的不确定性 C. 软件测试方法不完善 D. 软件程序效率不高

答案：A。教材明确指出导致软件危机的两方面原因：① 软件本身的复杂性；② 软件开发的方法和技术不合理、不成熟。选项 A 是核心内在原因之一。

【2010-2011 期中 A · 有答案】1. 产生软件危机的内在原因可以归纳为两方面 (C)

A. 软件难识别、看不到 B. 要求高智商、高资金 C. **软件生产本身复杂性 + 软件开发方法和技术** D. 软件难理解、硬件复杂
教材原文。

【2010-2011 期中 B】1. "软件危机"是指 ()

A. 计算机病毒的出现 B. 利用计算机进行经济犯罪活动 C. **软件开发和维护中出现的一系列问题** D. 人们过分迷恋计算机系统

答案：C。教材定义。

【2011-2012 期末 B · 有答案】1. 软件工程对于软件开发最主要的贡献是 (C)

A. 解决了软件危机的所有问题 B. 进一步提高了软件开发效率 C. **规范了软件开发的各项活动** D. 解决了软件项目管理的难题

A 错（未完全解决）；B 不是"最主要"贡献；D 太片面。软件工程最大的贡献是**将工程化原则引入开发，规范了软件开发的各项活动**。

【2018-2019 期末 B】1. 软件开发的 ____ 与软件产品的低质量之间的矛盾最终导致了软件危机。(A)

A. **高成本** B. 长周期 C. 高难度 D. 大规模

答案：A。教材原文："软件开发的高成本与软件产品的低质量之间的尖锐矛盾终于导致软件危机。"

【2024-2025 期中 A】1. 关于软件工程错误的说法是 ()

A. 软件工程规范了软件开发的各项活动 B. 软件内容涵盖软件的开发过程和管理过程 C. 软件工程知识是随着软件发展动态变化的 D. **如果不遵照软件工程方法，项目一定会失败**

答案：D。D 过于绝对。软件工程是最佳实践的系统化总结，不遵照不一定失败（小型项目可能成功），但风险显著增加。

【2024-2025 期中 A】2. 软件工程的出现解决了以下哪一个问题 ()

A. 程序设计方法 B. 项目管理 C. **软件开发过程的规范性** D. 需求经常性变动

答案：C。软件工程最大的贡献是规范了软件开发过程。需求变动是软件危机的成因之一，软件工程无法"解决"需求变动本身（只能通过模型控制）。

三、填空题（2024-2025 期中 A 卷）

1. 软件是包括 ()、数据和相关文档的完整集合。

答案：程序。教材软件定义。

2. 为了解决软件危机的问题，应该引入 () 的原则。

答案：工程化。Fritz Bauer 提出运用工程化原则和方法组织软件开发。

3. 工程的质量特性主要体现在计划、() 和应用处理四个方面。

计划、执行、检查、应用处理

4. 软件工程的项目必须要考虑这三个方面的因素：() 和**软件质量**。

答案：成本、进度。教材原文："在给定成本、进度的前提下，开发出满足用户需求.....的软件产品"。

四、简答题

【2009-2010 期中 A】1. 软件危机所暴露的问题时至今日仍然没有完全解决，请给出你所认为的最严重的问题是什么？并结合目前软件工程的进展，阐述你所认为最可能的解决方式。

参考解答：

最严重的问题——**软件需求的不确定性与持续变化**。原因：用户需求在开发初期难以确切表达，开发中又不断变化；修改引入新错误，导致可维护性差、质量难保证。

最可能的解决方式：

1. 采用迭代 / 增量 / 敏捷等适应变化的开发模型（如 UP、XP）；
2. 加强需求工程，引入需求确认、需求管理、变更控制机制；
3. 推行基于构件的复用与面向服务架构（SOA），降低耦合；
4. 强化项目管理（CMM / CMMI 等成熟度模型）；
5. 持续集成与自动化测试，提升质量保证能力。

【2010-2011 期中 B】1. 简述计算机系统的定义及六个组成元素？

参考解答：

计算机系统是由硬件子系统和软件子系统组成的、能按照用户要求处理信息的完整系统。

六个组成元素（结合硬件 + 软件分层）：

1. 处理器（CPU）
2. 内存（主存）
3. I / O 设备（输入输出设备）
4. 系统软件（操作系统、编译程序等）
5. 中间件（管理分布式资源与通信）
6. 应用软件（特定领域服务）

本题开放，可结合具体教材章节组织答案。

【2013–2014 期末 A · 有答案】1. 请列举出软件生命周期中贯穿的软件工程过程的六个基本活动（4 分）

参考答案：

1. 制定计划（含可行性分析、系统分析）；
2. 需求分析（需求定义）；
3. 软件设计（概要设计 + 详细设计）；
4. 程序编写 / 编码；
5. 软件测试；
6. 运行和维护。

【2015–2016 期中 A · 有答案】1. 请给出软件的定义。

参考答案：软件是包括程序、数据及其相关文档的完整集合。

【2015–2016 期中 A · 有答案】2. 请解释软件工程产生的历史背景。

参考答案：

1. 由于软件危机的产生导致软件工程概念的诞生；
2. 软件危机主要由于落后的软件开发方式无法满足快速增长软件需求；
3. 软件危机的 7 个主要表现（成本失控、质量低、不可维护、缺乏文档等）；
4. 1968 年 NATO 会议上 Fritz Bauer 首次提出"软件工程"概念，运用工程化原则和方法组织软件开发。

【2015–2016 期中 A · 有答案】4. 请给出软件工程的 6 项基本活动。

参考答案：制定计划；需求分析；软件设计；编码；软件测试；运行和维护。

【2018–2019 期末 B】5. 给出软件项目失败三个可能性。

参考答案：

1. 开发进度失控：不能在规定时间内成功完成项目开发；
2. 开发成本过高：导致软件项目无法收回成本；
3. 可维护性差：所开发项目缺乏可维护性，不能适应变化的需求，很快夭折。

可补充：需求不明、缺乏文档、团队管理不当等。

【2021–2022 期末 A】1. 根据软件危机的现象并结合本学期你在小组课程作业所遇到的问题，说明软件工程方法是否能解决软件开发过程中的问题。

参考答案：

1. 软件工程的價值：通过"方法+工具+过程"三要素系统化指导开发；通过 4 条基本原则控制需求易变性、规范设计、强化支撑、加强管理；通过 6 项基本活动覆盖软件生命周期；显著提升了软件质量和开发效率。
2. 局限：软件工程未能完全解决软件危机（行业共识）。
3. 结合课程作业：（举例）需求不清导致后期返工；小组成员协作中通信开销随人数 n 增大急剧上升 $(n(n-1)/2)$ ；版本控制不当导致代码冲突；测试覆盖不足导致上线后 bug。
4. 结论：软件工程方法部分解决了开发问题，但仍需结合实际不断改进——遵循 Boehm 第 7 原理"承认不断改进软件实践的意义"。

【2023–2024 期末 A】1. 请描述软件工程产生的历史背景及作用。

参考答案（详见"开放题答题模板 6"）：

历史背景：① 20 世纪 60 年代软件规模扩大、复杂度提高，落后的手工作坊式开发方式无法满足需求；② 软件危机爆发（IBM OS/360、火星探测器等典型案例）；③ 1968 年原联邦德国 NATO 会议，Fritz Bauer 首次提出"软件工程"概念。

作用：① 系统化指导（三要素）；② 规范化活动（6 项基本活动）；③ 质量保证（4 原则 + 8 一般原理 + Boehm 7 原理）；④ 知识体系（SWEBOOK）；⑤ 缓解危机（但未完全解决）。

A4 双栏 Cheat Sheet（考前速查）

◆ 软件定义与特征

- 软件 = 程序 + 数据 + 文档（IEEE / 公认）
- 软件 = 程序 + 文档（软件工程视角）
- 程序 = 算法 + 数据结构（结构化程序设计）
- 软件是逻辑实体，无机械磨损但有退化
- 特点 8 条：逻辑 / 无制造 / 退化 / 受约束 / 手工 / 复杂 / 高成本 / 社会因素

◆ 软件分类

- 按对象：通用 / 定制 / 可配置
- 按层次：系统 / 中间件 / 应用
- 按使用：单机 / 服务器 / 客户端 / SaaS
- 按工作方式：实时 / 分时 / 交互 / 批处理
- LOC 规模：微 / 小 / 中 / 大 / 甚大 / 极大

◆ 软件发展四阶段

- 程序设计（50–60）：个体手工，汇编
- 程序系统（60–70）：小组作坊，高级语言，软件危机出现
- 传统软件工程（70–90）：CASE、CMM、面向对象
- 现代软件工程（90 至今）：Internet、构件、SOA

◆ 软件危机

- 7 表现：计划难 / 费时失控 / 不满意 / 质量差 / 不可维护 / 无文档 / 成本升
- 原因：① 复杂性 ② 方法技术不合理
- 典型案例：IBM OS/360（5000 人年）；火星探测器（"，" vs "。"）

◆ 软件工程诞生

- 1968 年 Fritz Bauer（NATO 会议，原联邦德国）
- 核心思想：将工程化原则应用于软件
- IEEE 定义：系统化、规范化、量的方法

◆ 三要素

- 方法 / 工具 / 过程 (方工过)
- 方法 = 如何做; 工具 = CASE 支撑环境; 过程 = 综合应用

◆ 目标 10 属性

- 可修改 / 有效 / 可靠 / 可理解 / 可维护
- 可重用 / 可适应 / 可移植 / 可追踪 / 可互操作
- 前提: 成本 + 进度

◆ 6 项基本活动

- 计划 → 需求 → 设计 → 编码 → 测试 → 维护

◆ 4 条基本原则

- ① 适宜开发模型 ② 合适设计方法
- ③ 高质量工程支持 ④ 重视过程管理

◆ 8 条一般原理

- 抽象 / 信息隐藏 / 模块化 / 局部化
- 确定性 / 一致性 / 完备性 / 可验证性

◆ Boehm 7 基本原理 (计评控技查少进)

- ① 分阶段生命周期计划 ② 阶段评审 (63% 错误在编码前)
- ③ 严格产品控制 (基线配置) ④ 现代程序设计技术
- ⑤ 结果清楚审查 ⑥ 人员少而精 ($n(n-1)/2$) ⑦ 不断改进

◆ SWEBOK

- 5 目的: 一致观点 / 分界 / 内容 / 主题 / 认证基础
- 3 阶段: 草人 → 石人 → 铁人 (2004)
- 10 知识域:
 - 开发主流程: 需求 / 设计 / 构造 / 测试 / 维护
 - 支撑活动: 配置管理 / 工程管理 / 工程过程 / 工具方法 / 质量
- 相关学科: 计科、计工、数学、管理、项管、质管、软人类工、系统工程

◆ 答题口诀

- 退化曲线 = 锯齿形 (修改引入新错误)
- 硬件曲线 = 浴盆 (磨合 → 稳定 → 磨损)
- "唯一原因 / 一定能" → 多半是错误选项
- 失败主因: 管理不当 (非技术)